EXPERIMENT 10

Autoencoders for Image Denoising

AIM

To implement an **Autoencoder using TensorFlow/Keras** for image denoising and reconstruct clean images from noisy inputs.

THEORY

An **Autoencoder** is a neural network used for unsupervised learning.

It consists of:

- **Encoder** → compresses input into latent representation
- **Decoder** → reconstructs the input

In **image denoising**, the model learns to:
Remove noise and recover the original image

Mathematical Formulation

Autoencoder Function:

$$\hat{x} = g(f(x))$$

Where:

- (f(x)) = encoder
- (g(x)) = decoder
- (\hat{x}) = reconstructed image

DATASET SOURCE

Dataset: **MNIST Dataset**
Source: TensorFlow built-in

https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz

DATASET DESCRIPTION

- Images of handwritten digits (0–9)
- Total: 70,000 images
- Size: 28 × 28 pixels
- Used for denoising task

ALGORITHM LIMITATIONS

- May lose fine details
- Requires tuning of architecture
- Computationally expensive
- Needs sufficient data

METHODOLOGY / WORKFLOW

1. Import libraries
2. Load dataset
3. Normalize data
4. Add noise to images
5. Build autoencoder
6. Train model
7. Reconstruct images
8. Compare results

CODE IMPLEMENTATION

Step 1: Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Reshape
```

Step 2: Load Dataset

```
(X_train, _), (X_test, _) = mnist.load_data()
```

Step 3: Data Preprocessing

```
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten images
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)
```

Step 4: Add Noise

```
noise_factor = 0.5

X_train_noisy = X_train + noise_factor * np.random.normal(size=X_train.shape)
X_test_noisy = X_test + noise_factor * np.random.normal(size=X_test.shape)

# Clip values
X_train_noisy = np.clip(X_train_noisy, 0., 1.)
X_test_noisy = np.clip(X_test_noisy, 0., 1.)
```

Step 5: Build Autoencoder

```
input_img = Input(shape=(784,))

# Encoder
encoded = Dense(128, activation='relu')(input_img)
encoded = Dense(64, activation='relu')(encoded)

# Decoder
decoded = Dense(128, activation='relu')(encoded)
decoded = Dense(784, activation='sigmoid')(decoded)

autoencoder = Model(input_img, decoded)
```

Step 6: Compile Model

```
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
```

Step 7: Train Model

```
history = autoencoder.fit(
    X_train_noisy, X_train,
    epochs=10,
    batch_size=256,
    validation_data=(X_test_noisy, X_test)
```

```
)
```

Step 8: Denoise Images

```
decoded_imgs = autoencoder.predict(X_test_noisy)
```

Step 9: Visualization

```
n = 5
plt.figure(figsize=(10,4))

for i in range(n):
    # Noisy image
    ax = plt.subplot(3, n, i+1)
    plt.imshow(X_test_noisy[i].reshape(28,28), cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    # Reconstructed image
    ax = plt.subplot(3, n, i+n+1)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')
    plt.title("Denoised")
    plt.axis('off')

    # Original image
    ax = plt.subplot(3, n, i+2*n+1)
    plt.imshow(X_test[i].reshape(28,28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

plt.show()
```

<u>OUTPUT</u>

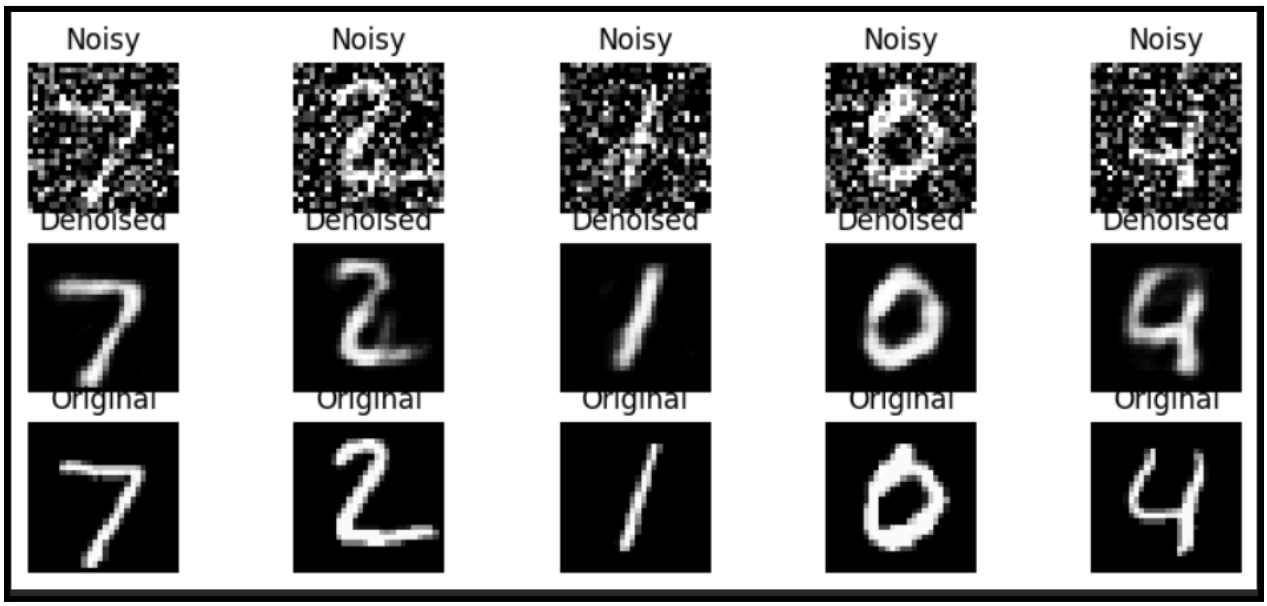
- Noisy images are cleaned effectively
- Reconstructed images are close to original
- Model shows good denoising capability

Screenshots:

- Training data

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
```

- Noisy vs Denoised vs Original images



CONCLUSION

In this experiment, an autoencoder was successfully implemented for image denoising.

The results show that:

- Autoencoders can effectively remove noise
- Neural networks can learn compressed representations
- Deep learning is powerful for image reconstruction

This experiment demonstrates the use of unsupervised learning in real-world image processing tasks.